



Indian Institute of Information Technology,
Design and Manufacturing, Kancheepuram

Advanced Memory Management in *xv6*

Memory Statistics • Demand Paging with `mmap` • Shared Memory IPC • Copy-on-Write Fork

Operating Systems Course Project Report

Team Members

Rohit Kumar	CS23B2053
Tarun Mamillapalli	CS23B2004
Suvam Sahu	CS23I1038

Course Faculty:

Dr. Santhanam Raghavan
Department of Computer Science and Engineering
IIITDM Kancheepuram

Contents

1	Part A: Memory Statistics	3
1.1	Introduction	3
1.2	System Call Specifications	3
1.2.1	Virtual Page Counter (<code>numvp()</code>)	3
1.2.2	Physical Page Counter (<code>numpp()</code>)	3
1.2.3	Page Table Size Calculator (<code>getptsize()</code>)	4
1.3	Testing and Validation	5
1.3.1	Test Program Structure	5
1.3.2	Baseline Memory Analysis	5
1.3.3	Dynamic Allocation Testing	6
2	Part B: Demand Paging	7
2.1	Introduction and Objectives	7
2.2	System Call Specifications	7
2.2.1	Memory Mapping System Call (<code>mmap()</code>)	7
2.2.2	Key Concepts Demonstrated	8
2.2.3	Difference from <code>sbrk()</code>	8
2.3	Testing and Validation	8
2.3.1	Test Program Structure	8
2.3.2	Test Scenarios	8
2.3.3	Test Results	10
2.3.4	Analysis of Results	10
2.4	Page Fault Handler for Demand Paging	10
2.4.1	Overview	10
2.4.2	Handler Implementation	11
2.4.3	Key Functions	11
2.4.4	Error Cases	11
3	Part C: Shared Memory	12
3.1	Introduction and Objectives	12
3.2	System Call Specifications	12
3.2.1	Shared Memory Allocation (<code>mapshared()</code>)	12
3.2.2	Shared Memory Lookup (<code>getshared()</code>)	13
3.2.3	Shared Memory Deallocation (<code>unmapshared()</code>)	14
3.3	Testing and Validation	15
3.3.1	Test Program Structure	15
3.3.2	Test Results	16
3.3.3	Analysis of Results	16
4	Part D: Copy-on-Write Fork	17
4.1	Key Features	17
4.2	Implementation Details	17
4.2.1	System Calls and Helper Functions	17
4.2.2	Reference Count Data Structures	18
4.2.3	Reference Count Operations	18
4.2.4	Fork Modification (<code>copyvm()</code>)	19
4.2.5	Page Fault Handler	20

4.3	Testing and Validation	22
4.3.1	Test Program	22
4.3.2	Test Results	23
5	Conclusion	25
5.1	Achievements Summary	25
5.2	Practical Implications	25
5.3	Final Remarks	25

1 Part A: Memory Statistics

1.1 Introduction

This section implements three system calls to provide runtime memory statistics for processes in xv6, enabling visibility into virtual memory usage, physical memory allocation, and page table overhead.

1.2 System Call Specifications

1.2.1 Virtual Page Counter (numvp())

The `numvp()` system call returns the total number of virtual pages allocated to a process's user address space. This count includes:

- Pages allocated for program code and data segments
- Dynamically allocated heap pages
- Stack guard page for overflow protection

The implementation calculates virtual pages using the process size stored in `struct proc`:

$$\text{Virtual Pages} = \left\lceil \frac{\text{Process Size (p->sz)}}{\text{Page Size}} \right\rceil + 1 \quad (1)$$

where the additional page accounts for the stack guard page.

```
// Returns the number of virtual pages in the user part of the address space, up to p->sz, plus the stack guard page.
int sys_numvp(void)
{
    struct proc *p = myproc();
    int numpages = 0;

    // Count pages from 0 up to the program size (p->sz)
    numpages = PGROUNDUP(p->sz) / PGSIZE;

    // Add one for the stack guard page, as required
    numpages += 1;

    return numpages;
}
```

Figure 1: Code snippet showing the `numvp()` system call implementation

1.2.2 Physical Page Counter (numpp())

The `numpp()` system call determines the actual number of physical RAM pages allocated to a process by traversing the process page table. Unlike virtual page counting, this function:

- Walks the two-level page table structure using `walkpgdir()`
- Validates each page table entry for the PTE_P (present) flag
- Counts only pages with valid physical memory assignments

```

// Returns the number of physical pages in the user part of the address space by walking the page table.
int sys_numpp(void)
{
    struct proc *p = myproc();
    pde_t *pgdir = p->pgdir;
    pte_t *pgtab;
    int count = 0;
    int i, j;

    // Loop through user-space Page Directory Entries and KERNBASE is the end of user space.
    for (i = 0; i < PDX(KERNBASE); i++)
    {
        if (pgdir[i] & PTE_P)
        {
            // PDE is present, get pointer to Page Table page
            pgtab = (pte_t *)P2V(PTE_ADDR(pgdir[i]));

            // Loop through all Page Table Entries in this page table
            for (j = 0; j < NPENTRIES; j++)
            {
                if (pgtab[j] & PTE_P)
                {
                    // PTE is present, meaning a physical page is mapped
                    count++;
                }
            }
        }
    }
    return count;
}

```

Figure 2: Code snippet showing the `numpp()` system call implementation

1.2.3 Page Table Size Calculator (`getptsize()`)

The `getptsize()` system call computes the total memory overhead consumed by the page table structure itself, including:

- One page directory (outer page table)
- All allocated inner page table pages
- Both user and kernel address space mappings

```

// Returns the size of the page table (in pages), including the page directory and all inner page table pages.
int sys_getptsize(void)
{
    struct proc *p = myproc();
    pde_t *pgdir = p->pgdir;

    // Start count at 1 for the page directory itself
    int count = 1;
    int i;

    // Loop through *all* Page Directory Entries (user and kernel)
    for (i = 0; i < NPENTRIES; i++)
    {
        if (pgdir[i] & PTE_P)
        {
            // This PDE points to a valid (present) page table page
            count++;
        }
    }
    return count;
}

```

Figure 3: Implementation of `getptsize()` including page table traversal

1.3 Testing and Validation

1.3.1 Test Program Structure

A dedicated test program was developed to validate the correctness of the memory statistics system calls. The program prints the virtual page count, physical page count, and page table size before and after invoking `sbrk()` for dynamic memory allocation.

```
1  #include "common/types.h"
2  #include "user/user.h"
3
4  int main(int argc, char *argv[])
5  {
6      printf(1, "Initial state:\n");
7      printf(1, "  Virtual Pages (numvp): %d\n", numvp());
8      printf(1, "  Physical Pages (numpp): %d\n", numpp());
9      printf(1, "  Page Table Size (getptsize): %d\n", getptsize());
10     printf(1, "You will see 1 page extra in virtual space (because of guard page)\n");
11
12     printf(1, "\nAllocating one page (4096 bytes) with sbrk...\n");
13     if (sbrk(4096) == (void *)-1)
14     {
15         printf(2, "sbrk failed!\n");
16     }
17     else
18     {
19         printf(1, "\nAfter sbrk(4096):\n");
20         printf(1, "  Virtual Pages (numvp): %d\n", numvp());
21         printf(1, "  Physical Pages (numpp): %d\n", numpp());
22         printf(1, "  Page Table Size (getptsize): %d\n", getptsize());
23         printf(1, "After sbrk, both virtual and physical page increased by 1\n");
24     }
25
26     exit();
27 }
```

Figure 4: Test program (`memStats.c`) used to evaluate memory statistics before and after `sbrk()` allocation

1.3.2 Baseline Memory Analysis

At program start, no dynamic memory has been allocated, and the memory layout reflects only the code, data, heap (empty), and stack segments.

```
init: starting sh
$ memStats
Initial state:
  Virtual Pages (numvp): 4
  Physical Pages (numpp): 3
  Page Table Size (getptsize): 66
You will see 1 page extra in virtual space (because of guard page)

Allocating one page (4096 bytes) with sbrk...

After sbrk(4096):
  Virtual Pages (numvp): 5
  Physical Pages (numpp): 4
  Page Table Size (getptsize): 66
After sbrk, both virtual and physical page increased by 1
$ _
```

Figure 5: Output showing memory statistics before and after `sbrk()` allocation

Key Observations (Before sbrk()):

- **Virtual Pages = 4:** This includes three pages for the program (code, data, and stack) and one stack guard page.
- **Physical Pages = 3:** Only the program pages have physical frames; the guard page is not backed by physical memory.
- **Page Table Size = 66:** Represents the total memory used for page directory and page table pages mapping the process's address space.

1.3.3 Dynamic Allocation Testing

Next, a call to `sbrk(4096)` is performed, requesting one additional 4 KB page for the heap.

Key Observations (After sbrk(4096)):

- **Virtual Pages: 4 → 5** A new virtual page appears in the heap region.
- **Physical Pages: 3 → 4** The newly allocated heap page is immediately backed by a physical frame.
- **Page Table Size: 66 (unchanged)** No new page table pages were required; the existing page tables already had space to map this additional page.



Figure 6: Memory layout diagram showing address space organization

2 Part B: Demand Paging

2.1 Introduction and Objectives

This section implements a simplified version of the `mmap` system call that introduces demand paging to xv6. Unlike standard xv6 where `sbrk()` immediately allocates both virtual and physical memory, `mmap` demonstrates lazy allocation by creating virtual memory mappings without corresponding physical pages until they are accessed.

2.2 System Call Specifications

2.2.1 Memory Mapping System Call (`mmap()`)

The `mmap()` system call implements demand paging by creating virtual memory mappings without immediately allocating physical pages. This lazy allocation approach defers physical memory commitment until the pages are actually accessed by the process.

```
int
sys_mmap(void)
{
    int nbytes;
    struct proc *p = myproc();

    if(argint(0, &nbytes) < 0)
        return 0; // Return 0 on invalid input

    if([nbytes <= 0 || (nbytes % PGSIZE) != 0])
        return 0; // Must be positive and a multiple of PGSIZE

    // Store the starting virtual address of the new region
    uint old_sz = p->sz;

    // Grow the virtual address space (but not physical memory)
    if (p->sz + nbytes > KERNBASE)
        return 0; // Don't grow into the kernel

    p->sz += nbytes;

    // Return the starting virtual address of the new region
    return old_sz;
}
```

Figure 7: Implementation of the `mmap()` system call for demand paging

Key Points:

- **Input Validation:** Checks if `nbytes` is positive and page-aligned (multiple of 4096)
- **Virtual Expansion Only:** Updates `p->sz` (process size) but doesn't allocate physical memory
- **Boundary Check:** Ensures we don't grow into kernel space (`KERNBASE`)
- **Returns Starting Address:** The old `p->sz` becomes the start of the new mapped region

2.2.2 Key Concepts Demonstrated

- **Lazy Allocation:** `mmap` creates virtual mappings but defers physical allocation
- **Page Fault Driven:** Physical allocation happens through page fault mechanism
- **Memory Efficiency:** Only actually accessed pages consume physical RAM
- **Granular Control:** Each 4KB page allocated independently as needed

2.2.3 Difference from `sbrk()`

The fundamental difference between `mmap()` and `sbrk()` lies in their allocation strategy:

- `sbrk(4096)`: Virtual +1, Physical +1 (immediately)
- `mmap(4096)`: Virtual +1, Physical +0 initially, Physical +1 on first access

This delayed allocation provides significant memory efficiency for sparse or large memory regions.

2.3 Testing and Validation

2.3.1 Test Program Structure

A comprehensive test program was developed to validate the demand paging functionality. The program tests various scenarios including error conditions, single page allocation, and multiple page allocation with sparse access patterns.

The test program demonstrates the core demand paging mechanism:

What happens:

1. `mmap(4096)` increases virtual pages by 1, physical pages stay the same
2. `addr[0] = 'a'` tries to write to unmapped memory → triggers a page fault
3. Page fault handler allocates physical memory and maps it
4. Physical pages increase by 1

2.3.2 Test Scenarios

The test program validates the following scenarios:

1. **Error Case 1:** Negative size input (-1234 bytes)
2. **Error Case 2:** Non-page-aligned size (1234 bytes)
3. **Case 3:** Single page allocation (4096 bytes)
 - Memory statistics after `mmap()` call
 - Memory statistics after first access
4. **Case 4:** Two-page allocation (8192 bytes)
 - Memory statistics after `mmap()` call
 - Memory statistics after accessing first page
 - Memory statistics after accessing second page

```

int main(void)
{
    uint ret;
    printf(1, "Start: memory usage in pages: virtual: %d, physical %d\n", numvp(), numpp());

    ret = mmap (-1234);
    if(ret == 0)
        printf(1, "mmap failed for wrong inputs\n");
    else
        exit();

    ret = mmap (1234);
    if(ret == 0)
        printf(1, "mmap failed for wrong inputs\n");
    else
        exit();

    ret = mmap(4096);

    if(ret == 0 )
        printf(1, "mmap failed\n");
    else {
        printf(1, "After mmap one page: memory usage in pages: virtual: %d, physical %d\n", numvp(), numpp());

        char *addr = (char *) ret;

        addr[0] = 'a';

        printf(1, "After access of one page: memory usage in pages: virtual: %d, physical %d\n", numvp(), numpp());
    }

    ret = mmap(8192);

    if(ret == 0 )
        printf(1, "mmap failed\n");
    else {
        printf(1, "After mmap two pages: memory usage in pages: virtual: %d, physical %d\n", numvp(), numpp());

        char *addr = (char *) ret;

        addr[0] = 'a';

        printf(1, "After access of first page: memory usage in pages: virtual: %d, physical %d\n", numvp(), numpp());
        addr[8000] = 'a';

        printf(1, "After access of second page: memory usage in pages: virtual: %d, physical %d\n", numvp(), numpp());
    }

    exit();
}

```

Figure 8: Test program (mmap_test.c) for validating demand paging functionality

2.3.3 Test Results

The test program execution demonstrates the demand paging behavior in action:

```
init: starting sh
$ test-mmap
Start: memory usage in pages: virtual: 4, physical 3
mmap failed for wrong inputs
mmap failed for wrong inputs
After mmap one page: memory usage in pages: virtual: 5, physical 3
After access of one page: memory usage in pages: virtual: 5, physical 4
After mmap two pages: memory usage in pages: virtual: 7, physical 4
After access of first page: memory usage in pages: virtual: 7, physical 5
After access of second page: memory usage in pages: virtual: 7, physical 6
```

Figure 9: Output of the `mmap` test program showing demand paging in action

2.3.4 Analysis of Results

Key Observations from Test Output:

- **Error Handling:** The system correctly rejects invalid inputs:
 - Negative size: Returns 0 (failure)
 - Non-page-aligned size: Returns 0 (failure)
- **Single Page Test:**
 - After `mmap(4096)`: Virtual pages increase by 1, Physical pages unchanged
 - After accessing `addr[0]`: Physical pages increase by 1 (page fault triggered)
- **Two-Page Test:**
 - After `mmap(8192)`: Virtual pages increase by 2, Physical pages unchanged
 - After accessing `addr[0]`: Physical pages increase by 1 (first page fault)
 - After accessing `addr[8000]`: Physical pages increase by 1 (second page fault)
- **Memory Efficiency:** Only accessed pages consume physical memory, demonstrating optimal resource utilization.

2.4 Page Fault Handler for Demand Paging

2.4.1 Overview

The page fault handler is the critical component that enables demand paging for the `mmap()` system call. When a process accesses memory that was mapped by `mmap()` but not yet allocated, a page fault occurs, and this handler transparently allocates the physical page.

2.4.2 Handler Implementation

```
case T_PGFLT: {
    uint va = rcr2(); // Faulting address
    struct proc *p = myproc();
    uint a = PGROUNDDOWN(va);
    pte_t *pte;

    // Check if address is valid
    if (va >= p->sz) {
        cprintf("seg fault at va %p\n", va);
        p->killed = 1;
        break;
    }

    // Get page table entry
    pte = walkpgdir(p->pgdir, (char*)a, 0);

    // Demand paging case: page not present
    if(!pte || !(*pte & PTE_P)) {
        // Allocate physical page
        if (allocuvn(p->pgdir, a, a + PGSIZE) == 0) {
            cprintf("OOM at va %p\n", va);
            p->killed = 1;
        }
    }

    // Flush TLB
    switchvm(p);
    break;
}
```

Figure 10: Page fault handler for demand paging in `trap.c`

2.4.3 Key Functions

- `rcr2()`: Reads the faulting virtual address from the CR2 register. This is the address that caused the page fault.
- `PGROUNDDOWN(va)`: Aligns the address to the nearest lower page boundary (4KB aligned). Converts address to page start.
- `walkpgdir()`: Walks the two-level page table to find the Page Table Entry (PTE) for the given virtual address.
- `allocuvn()`: Allocates a physical page, maps it to the virtual address, and initializes it to zero.
- `switchvm()`: Reloads the CR3 register with the process's page directory address, flushing TLB entries.

2.4.4 Error Cases

- **Segmentation fault**: Occurs when the faulting address $\geq p\text{->}sz$, meaning the process accessed memory outside its allocated address space.
- **Out of memory**: When `allocuvn()` returns 0, indicating no free physical pages are available for allocation.

3 Part C: Shared Memory

3.1 Introduction and Objectives

This section implements a shared memory mechanism in xv6 that allows a page allocated by a parent process to be inherited by its child process, with both processes mapping to the same physical memory. This enables efficient inter-process communication (IPC) by avoiding data copying between processes.

3.2 System Call Specifications

3.2.1 Shared Memory Allocation (mapshared())

The mapshared() system call allocates one page of physical memory and maps it at the end of the process's address space with the PTE_S flag set.

```
// Maps one shared page at the end of the process's address space.
int sys_mapshared(void)
{
    struct proc *p = myproc();
    char *mem;
    uint va = p->sz; // New page will be at the current end

    // Allocate one page of physical memory
    if ((mem = kalloc()) == 0)
        return -1;

    // Zero out the page
    memset(mem, 0, PGSIZE);

    // Map the page in the page table with the Shared flag
    // We must add PTE_P, PTE_W, PTE_U, and our new PTE_S flag
    if (mappages(p->pgdir, (void *)va, PGSIZE, V2P(mem), PTE_W | PTE_U | PTE_P | PTE_S) < 0)
    {
        kfree(mem); // Free the page if mapping fails
        return -1;
    }

    // Grow the process size to include the new page
    p->sz += PGSIZE;

    // Return the starting virtual address of the mapped page
    return va;
}
```

Figure 11: Implementation of sys_mapshared() system call

Key Steps:

1. Allocate physical page using kalloc()
2. Zero-initialize the page for security
3. Map the page with PTE_W | PTE_U | PTE_P | PTE_S flags
4. Grow process size to include new page
5. Return virtual address of mapped page

3.2.2 Shared Memory Lookup (getshared())

The `getshared()` system call scans the process's page table to find and return the virtual address of the page marked with the `PTE_S` flag.

```
// Returns the virtual address of the shared page.
int sys_getshared(void)
{
    struct proc *p = myproc();
    pde_t *pgdir = p->pgdir;
    pte_t *pte;
    uint va;

    // Iterate through the user virtual address space
    for (va = 0; va < p->sz; va += PGSIZE)
    {
        // Find the page table entry
        pte = walkpgdir(pgdir, (void *)va, 0);

        if (pte == 0) // Page directory entry not present
            continue;

        // Check if the page is present and has the shared flag
        if ((*pte & PTE_P) && (*pte & PTE_S))
        {
            return va; // Found it
        }
    }

    return -1; // Not found
}
```

Figure 12: Implementation of `sys_getshared()` system call

Key Steps:

1. Iterate through virtual address space in page increments
2. Use `walkpgdir()` to get page table entry
3. Check if page has both `PTE_P` and `PTE_S` flags
4. Return virtual address when found, -1 if not found

3.2.3 Shared Memory Deallocation (unmapshared())

The unmapshared() system call unmaps the shared page and frees the corresponding physical memory.

```
// Unmaps the shared page.
int sys_unmapshared(void)
{
    struct proc *p = myproc();
    pde_t *pgdir = p->pgdir;
    pte_t *pte;
    uint va;
    char *pa;

    // Find the shared page's virtual address
    int shared_va = sys_getshared();
    if (shared_va == -1)
        return -1; // No shared page to unmap

    va = (uint)shared_va;

    // Get the PTE
    pte = walkpgdir(pgdir, (void *)va, 0);
    if (pte == 0 || !(*pte & PTE_P) || !(*pte & PTE_S))
        return -1; // Page not found or not shared

    // Get the physical address
    pa = (char *)P2V(PTE_ADDR(*pte));

    // Free the physical memory
    kfree(pa);

    // Invalidate the PTE
    *pte = 0;

    // If the shared page is at the end of memory, shrink proc size
    if (va == PGROUNDOWN(p->sz - 1))
    {
        p->sz -= PGSIZE;
    }

    // Flush the TLB
    switchvm(p);

    return 0; // Success
}
```

Figure 13: Implementation of sys_unmapshared() system call

Key Steps:

1. Use getshared() to find shared page
2. Get physical address from page table entry
3. Free physical memory using kfree()
4. Invalidate the PTE
5. Adjust process size if page was at end
6. Flush TLB for consistency

3.3 Testing and Validation

3.3.1 Test Program Structure

A comprehensive test program was developed to validate shared memory functionality between parent and child processes. The program uses synchronized writes and reads to verify that both processes share the same physical memory.

```
int main(int argc, char *argv[])
{
    uint *ap = (uint *) mapshared();
    int pid = fork();

    if(pid == 0) {
        sleep(2); //wait for parent to write
        uint *ac = (uint *) getshared();

        printf(1, "[Child] Reading from shared memory: %d\n", *ac);
        printf(1, "child %d\n", *ac);

        printf(1, "[Child] Writing 53 to shared memory\n");
        *ac = 53;

        sleep(10);

        printf(1, "[Child] Reading from shared memory: %d\n", *ac);
        printf(1, "child again %d\n", *ac);

        printf(1, "[Child] Writing 54 to shared memory\n");
        *ac = 54;

        sleep(20); //sleep to give parent time to unmap
    }
    else {
        printf(1, "[Parent] Writing 42 to shared memory\n");
        *ap = 42;

        sleep(5); //wait for child to reply

        printf(1, "[Parent] Reading from shared memory: %d\n", *ap);
        printf(1, "parent %d\n", *ap);

        printf(1, "[Parent] Writing 43 to shared memory\n");
        *ap = 43; //write again

        sleep(10);

        printf(1, "[Parent] Reading from shared memory: %d\n", *ap);
        printf(1, "parent again %d\n", *ap);

        if(unmapshared() < 0)
            printf(1, "could not unmap shared page\n");
        wait();
    }

    exit();
}
```

Figure 14: Test program (sharedMem.c) for validating shared memory functionality

Test Program Logic:

1. Parent allocates shared memory using `mapshared()`
2. Parent forks to create child process
3. Both processes synchronize using `sleep()` calls
4. Parent writes value (42), child reads and verifies

5. Child writes value (53), parent reads and verifies
6. Multiple read-write cycles to confirm persistence
7. Parent unmaps shared memory before exit

3.3.2 Test Results

The test program demonstrates successful shared memory communication between parent and child processes.

```
init: starting sh
$ sharedMem
[Parent] Writing 42 to shared memory
[Child] Reading from shared memory: 42
child 42
[Child] Writing 53 to shared memory
[Parent] Reading from shared memory: 53
parent 53
[Parent] Writing 43 to shared memory
[Child] Reading from shared memory: 43
child again 43
[Child] Writing 54 to shared memory
[Parent] Reading from shared memory: 54
parent again 54
$
```

Figure 15: Output of the shared memory test program showing successful IPC

3.3.3 Analysis of Results

Key Observations from Test Output:

1. Initial Write-Read:

- Parent writes value 42 to shared memory
- Child reads value 42 correctly → Confirms inheritance

2. Child Write-Parent Read:

- Child writes value 53 to same memory location
- Parent reads value 53 correctly → Confirms bidirectional sharing

3. Multiple Cycles:

- Parent writes 43, child reads 43
- Child writes 54, parent reads 54
- Demonstrates persistent shared state across multiple operations

4. Proper Cleanup:

- `unmapshared()` called successfully
- No memory leaks or double-free issues

4 Part D: Copy-on-Write Fork

This section implements Copy-on-Write (COW) functionality in xv6's `fork()` system call. COW allows parent and child processes to share physical pages initially, with private copies created only when a process attempts to write to a shared page. This optimization reduces memory usage and improves forking performance.

4.1 Key Features

- **Lazy Copying:** Physical pages are copied only on first write
- **Reference Counting:** Track how many processes share each page
- **Page Fault Detection:** Detect write attempts to read-only COW pages
- **Memory Efficiency:** Reduce memory duplication during forking

4.2 Implementation Details

4.2.1 System Calls and Helper Functions

The `getNumFreePages()` system call was added to monitor available memory. This helps track memory usage during COW operations.

```
//returns the number of free pages tracked in kalloc.c
extern int get_free_pages_count(void);

int
sys_getNumFreePages(void)
{
    return get_free_pages_count();
}
```

(a) `getNumFreePages()` system call

```
int get_free_pages_count(void) {
    acquire(&kmem.lock);
    int f = kmem_free_pages;
    release(&kmem.lock);
    return f;
}

int num_free_pages() {
    struct run *r;
    int count = 0;

    acquire(&kmem.lock);
    for(r = kmem.freelist; r; r = r->next)
        count++;
    release(&kmem.lock);

    return count;
}
```

(b) System call's Helper Functions

Figure 16: Functions

4.2.2 Reference Count Data Structures

Global data structures track reference counts for physical pages and monitor free memory availability.

```
// Maximum physical pages
#define MAX_PHY_PAGES ((PHYSTOP - KERNBASE) / PGSIZE)

// Reference count per physical page
static int phys_refcount[MAX_PHY_PAGES];

// Track free pages
static int kmem_free_pages = 0;
```

Figure 17: Reference count data structures in `kalloc.c`

4.2.3 Reference Count Operations

Three core functions manage page sharing: increment count when pages are shared, decrement when unmapped, and check current reference counts.

```
void incr_refcount(uint pa) {
    uint idx = (pa - KERNBASE) / PGSIZE;
    if (idx >= MAX_PHY_PAGES)
        panic("incr_refcount: out of range");

    acquire(&kmem.lock);
    phys_refcount[idx]++;
    release(&kmem.lock);
}
```

(a) `incr_refcount()`

```
void decr_refcount(uint pa) {
    uint idx = (pa - KERNBASE) / PGSIZE;
    if (idx >= MAX_PHY_PAGES)
        panic("decr_refcount: out of range");

    acquire(&kmem.lock);

    if (phys_refcount[idx] <= 0)
        panic("decr_refcount: underflow");

    phys_refcount[idx]--;
    int now = phys_refcount[idx];

    release(&kmem.lock);

    // Only free if no references
    if (now == 0) {
        kfree((char*)P2V(pa));
    }
}
```

(b) `decr_refcount()`

```
int get_refcount(uint pa) {
    uint idx = (pa - KERNBASE) / PGSIZE;
    if (idx >= MAX_PHY_PAGES) return -1;

    acquire(&kmem.lock);
    int r = phys_refcount[idx];
    release(&kmem.lock);

    return r;
}
```

(c) `get_refcount()`

Figure 18: Reference count management functions

Key Functions Implemented:

- `incr_refcount()` - Increment page reference count when allocated and shared
- `decr_refcount()` - Decrement count and free page if count reaches 0
- `get_refcount()` - Get current reference count for a page
- `get_free_pages_count()` - Track free pages for testing memory efficiency

4.2.4 Fork Modification (copyuvm())

The `copyuvm()` function was fundamentally modified to implement COW semantics during process forking. Instead of immediately copying all memory, it now shares pages between parent and child.

```
// Given a parent process's page table, create a copy for a child.
pde_t *copyuvm(pde_t *pgdir, uint sz)
{
    pde_t *d;
    pte_t *pte;
    uint pa, i, flags;

    // Create new page directory for child
    if ((d = setupkvm()) == 0)
        return 0;

    for (i = 0; i < sz; i += PGSIZE)
    {
        // Get parent's page table entry
        if ((pte = walkpgdir(pgdir, (void *)i, 0)) == 0)
            panic("copyuvm: pte should exist");
        if (!(pte & PTE_P))
            panic("copyuvm: page not present");

        pa = PTE_ADDR(*pte); // Physical address
        flags = (*pte) & 0xFFF; // Page flags
        uint perm = flags & ~PTE_P;

        // Get child's page table entry
        pte_t *cpte = walkpgdir(d, (void *)i, 1);
        if(cpte == 0) goto bad;

        if(flags & PTE_S) {
            // This is a shared memory page
            // Map SAME physical page in child's page table
            *cpte = pa | perm | PTE_P;
            incr_refcount(pa);
        } else {
            // This is a normal page - implement COW
            uint child_perm = perm & ~PTE_W; // Mark READ-ONLY

            // Map child to same physical page (read-only)
            *cpte = pa | child_perm | PTE_P;

            // Mark parent's page as read-only too
            *pte = pa | child_perm | PTE_P;

            // Increment reference count (now shared)
            incr_refcount(pa);
        }
    }

    // Flush TLB for parent after changing its PTEs
    lcr3(V2P(pgdir));
    return d;

bad:
    freevm(d);
    return 0;
}
```

Figure 19: Modified `copyuvm()` function implementing COW fork

Key Changes in Fork Behavior:

- **Create new page directory:** Sets up child's address space structure
- **No physical copying:** Maps child to parent's physical pages directly

- **Read-only mappings:** Marks both parent and child pages as read-only
- **Reference counting:** Increments count for all shared pages
- **TLB flush:** Updates translation lookaside buffer after modifications

COW Fork Process:

1. Create new page directory for child process
2. For each parent page:
 - Don't allocate new physical memory
 - Map SAME physical page in child's page table
 - Mark BOTH parent and child pages as READ-ONLY
 - Increment reference count (`incr_refcount()`)
3. Flush TLB to apply permission changes
4. Return child's page directory

4.2.5 Page Fault Handler

The trap handler was modified to detect and handle COW page faults when processes attempt to write to read-only shared pages.

```
trap(struct trapframe *tf)

case T_PGFLT: {
    uint va = rcr2(); // Get the faulting virtual address
    struct proc *p = myproc();
    uint a = PGROUNDOWN(va);
    pte_t *pte;

    // Check if address is valid
    if (va >= p->sz) {
        // Real segmentation fault
        cprintf("pid %d %s: seg fault at va %p\n", p->pid, p->name, va);
        p->killed = 1;
        goto trap_end;
    }

    // At this point, va < p->sz, so the address is valid.
    pte = walkpgdir(p->pgdir, (char*)a, 0);

    // Check if this is a Copy-on-Write fault
    // Page is present, but marked read-only
    if (!(*pte & PTE_W) && (*pte & PTE_U)) {
        uint pa = PTE_ADDR(*pte);
        int rc = get_refcount(pa);

        if (rc > 1) {
            // Shared page -> must copy (True COW case)
            char *mem = kalloc();
            if (mem == 0) {
                cprintf("pid %d %s: COW kalloc failed\n", p->pid, p->name);
                p->killed = 1;
                goto trap_end;
            }
            memmove(mem, (char*)P2V(pa), PGSIZE);
            decr_refcount(pa);
            *pte = V2P(mem) | PTE_P | PTE_W | PTE_U;
        }
        else {
            // Unique owner -> just make page writable
            *pte |= PTE_W;
        }
    }

    // Flush the TLB by reloading CR3
    switchvm(p);
    break;
}
```

Figure 20: COW fault handling in trap handler

COW Fault Handling Process:

1. **Page fault occurs** when process writes to read-only shared page
2. **Handler checks:** Is this a COW page? (read-only, refcount > 1)
3. **If YES (Shared page, refcount > 1):**
 - Allocate NEW physical page
 - Copy original page content to new page
 - Update page table to point to new page
 - Make new page WRITABLE
 - Decrease refcount of original page
4. **If NO (Unique owner, refcount = 1):**
 - Just restore write permission
 - No copying needed
5. **Flush TLB** to apply changes

Key Conditions Detected:

- `!(*pte & PTE_W)` - Page is read-only
- `(*pte & PTE_U)` - User page (not kernel)
- `get_refcount(pa) > 1` - Multiple processes share the page

4.3 Testing and Validation

4.3.1 Test Program

A comprehensive test program validates COW functionality by tracking free page counts before and after fork operations.

```
int main(void)
{
    printf(1, "=== COW Fork Test ===\n\n");

    // Track memory usage
    int free_before = getNumFreePages();
    printf(1, "1. Free pages before allocation: %d\n", free_before);

    // Allocate 4 pages (16KB)
    char *mem = sbrk(4 * 4096);
    printf(1, "2. After allocating 4 pages: %d\n", getNumFreePages());

    // Fill with data
    for (int i = 0; i < 4 * 4096; i++) {
        mem[i] = i % 256;
    }

    int before_fork = getNumFreePages();
    printf(1, "3. Before fork: %d\n", before_fork);

    int pid = fork();

    if (pid == 0) {
        // Child process
        printf(1, "\n--- Child Process ---\n");
        printf(1, "4. Child after fork (COW): %d\n", getNumFreePages());

        // Write to first page - triggers COW
        mem[0] = 0xFF;
        printf(1, "5. After writing to page 0: %d\n", getNumFreePages());

        // Write to second page - triggers COW
        mem[4096] = 0xAA;
        printf(1, "6. After writing to page 1: %d\n", getNumFreePages());

        // Leave pages 2-3 read-only (shared)
        printf(1, "7. Pages 2-3 remain shared\n");

        exit();
    } else {
        // Parent process
        wait();

        printf(1, "\n--- Parent Process ---\n");
        printf(1, "8. Parent after child exit: %d\n", getNumFreePages());

        // Write to third page - should trigger COW
        mem[8192] = 0xBB;
        printf(1, "9. After writing to page 2: %d\n", getNumFreePages());

        // Fourth page remains shared (never written by either)

        int free_after = getNumFreePages();
        printf(1, "\n=== Summary ===\n");
        printf(1, "Initial free pages: %d\n", free_before);
        printf(1, "Final free pages: %d\n", free_after);
        printf(1, "Memory saved by COW: %d pages\n",
            (4 * 2) - (free_before - free_after)); // 4 pages * 2 processes - actual usage
    }

    exit();
}
```

Figure 21: COW test program (cowtest.c)

4.3.2 Test Results

The test output demonstrates COW memory savings by showing page allocations only occur on write access.

```
$ cowtest
=== COW Fork Test ===

1. Free pages before allocation: 56221
2. After allocating 4 pages: 56217
3. Before fork: 56217

--- Child Process ---
4. Child after fork (COW): 56149
5. After writing to page 0: 56148
6. After writing to page 1: 56147
7. Pages 2-3 remain shared

--- Parent Process ---
8. Parent after child exit: 56217
9. After writing to page 2: 56217

=== Summary ===
Initial free pages: 56221
Final free pages: 56217
Memory saved by COW: 4 pages
$
```

Figure 22: COW test program output showing memory efficiency

Analysis of Test Output:

1. Initial State:

- Free pages: 56221 (baseline)
- System has ample available memory

2. After Allocating 4 Pages:

- Free pages: 56217 (decreased by 4)
- **Confirmed:** 4 pages allocated for parent's data

3. After Fork (Key COW Test):

- Free pages: 56149 (decreased by 68 from step 2)
- **Observation:** This decrease is NOT due to data copying
- **Explanation:** 68 pages used for child's page tables and process structure
- **Success:** No additional 4 pages copied → COW working

4. Child Writes to Page 0:

- Free pages: 56148 (decreased by 1)
- **Confirmed:** First COW copy triggered (page 0)

5. Child Writes to Page 1:

- Free pages: 56147 (decreased by 1)
- **Confirmed:** Second COW copy triggered (page 1)

6. Pages 2-3 Remain Shared:

- No page allocation for pages 2-3
- **Success:** Unwritten pages stay shared

7. After Child Exit:

- Free pages: 56217 (returned to pre-fork level +4)
- **Confirmed:** Child's page tables freed (68 pages)
- Child's COW copies (2 pages) also freed
- Parent's original 4 pages remain allocated

8. Parent Writes to Page 2:

- Free pages: 56217 (unchanged)
- **Explanation:** Page 2 was unique after child exit
- No COW needed - just write permission restored

Memory Overhead Explanation: The 68-page decrease after fork represents:

- Child process control block
- Page directory and page tables
- Kernel stack and other metadata
- **Not data pages** - confirming COW success

5 Conclusion

This project successfully implemented four advanced memory management features in the xv6 operating system, each demonstrating key principles of modern operating system design and optimization.

5.1 Achievements Summary

- **Memory Statistics:** Implemented three system calls (`numvp()`, `numpp()`, and `getptsize()`) that provide real-time visibility into process memory usage, enabling detailed monitoring of virtual memory, physical memory allocation, and page table overhead.
- **Demand Paging:** Developed a simplified `mmap()` system call that introduces lazy memory allocation, demonstrating how physical memory commitment can be deferred until actual page access, significantly improving memory efficiency for sparse memory usage patterns.
- **Shared Memory IPC:** Created a shared memory mechanism with three new system calls (`mapshared()`, `getshared()`, and `unmapshared()`) that enables zero-copy inter-process communication through inheritable shared pages marked with the `PTE_S` flag.
- **Copy-on-Write Fork:** Modified the `fork()` system call to implement COW semantics, reducing memory duplication during process creation by sharing physical pages between parent and child processes until write operations necessitate private copies.

5.2 Practical Implications

The implemented features have important practical applications:

- **Performance Improvement:** COW fork significantly reduces process creation time for memory-intensive applications.
- **Resource Optimization:** Demand paging enables efficient handling of large, sparsely accessed memory regions.
- **IPC Enhancement:** Shared memory provides high-speed communication between cooperating processes.
- **Debugging Support:** Memory statistics aid in performance analysis and memory leak detection.

5.3 Final Remarks

The successful implementation of these memory management features demonstrates a deep understanding of operating system internals, virtual memory concepts, and system programming. Each component was thoroughly tested and validated, showing correct behavior under various scenarios. This project not only enhances xv6's capabilities but also provides valuable insights into the design and implementation of memory management subsystems in modern operating systems.

The code is modular, well-documented, and serves as an educational resource for understanding advanced memory management techniques in practice.